

Margaret Meyer

CS 230 Project Software Design

Version 3.0

Table of Contents

CS 230 Project Software Design Template 1

Table of Contents 2
Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 4
Domain Model 4
Evaluation 5
Recommendations 8

Document Revision History

Version	Date	Author	Comments
1.0	09/20/2026	Margaret Meyer	Mapped out the different OS systems and how they function followed by which one is best for the task at hand.
2.0	10/01/2026	Margaret Meyer	Combed over the information provided and fix minor grammatical or spelling errors.
3.0	10/18/2026	Margaret Meyer	Proofread for info accuracy and grammatical errors

Instructions
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Executive Summary

Client: The Gaming Room
Project: Draw It or Lose it (Web Based Platform Expansion)

The Gaming Room is looking to transition its popular mobile app, *Draw It or Lose It,* from an android exclusive app to a web-based game accessible across multiple platforms and devices. To support multiuser gameplay, the application must manage games, teams, and players while ensuring data integrity and uniqueness across active sessions. Creative Technology Solutions proposes an object-oriented Java architecture that leverages established software design patterns specifically the Singleton and Iterator patterns to centralize game statement, guarantee unique identification for all entities, and enforce unique naming constraints across games and teams.

Requirements

The Gaming Room requires expanding its existing Android only mobile app into a web based multiuser gaming application accessible across multiple operating platforms and mobile devices.

- Business Requirements:
    - Expand market reach and revenue by offering cross platform accessibility for web browser and mobile use.
    - Support multiuser competitive play featuring one or more teams with multiple players assigned per team.
    - Maintain data integrity and prevent user confusion by enforcing unique game and team names across active sessions.
- Technical Requirements:
    - Centralize game state management so that only a single instance of the game service exists in memory at any given time.
    - Assign unique auto incrementing identifiers to every instance of a Game, Team, and Player.
    - Implement an extensible object-oriented architecture that allows seamless state checking prior to instantiating new entities.

Design Constraints

- Centralized Single Instance State:
  Developing in a web based distributed environment requires a single source of truth for managing global game sessions. Allowing multiple instances of the core service class would cause data fragmentation, duplicate ID assignments, and state desynchronization across client connections. To address this the application design is constrained to use the Singleton Pattern for GameService ensuring exactly one instance exists in server memory to manage entity creation and lookup routines.
- Unique Entity Naming and Verification:
  Players must be able to create and join games or teams using distinct names without complications. Because clients connect concurrently in a distributed environment the application must verify name availability before instantiating objects. This constrains method logic to employ the Iterator Pattern sequentially traversing active entity collections to check for existing name matches before allocating new IDs.

- Distributed Architecture and Cross Platform Compatibility:
  Transitioning from a native Android application to a web based distributed model constrains client server interaction. The backend code must decouple business logic from specific client operating systems. Utilizing an abstract base class and standard communication protocols ensures that any web browser or mobile client can consume server data without requiring OS specific backend modifications.

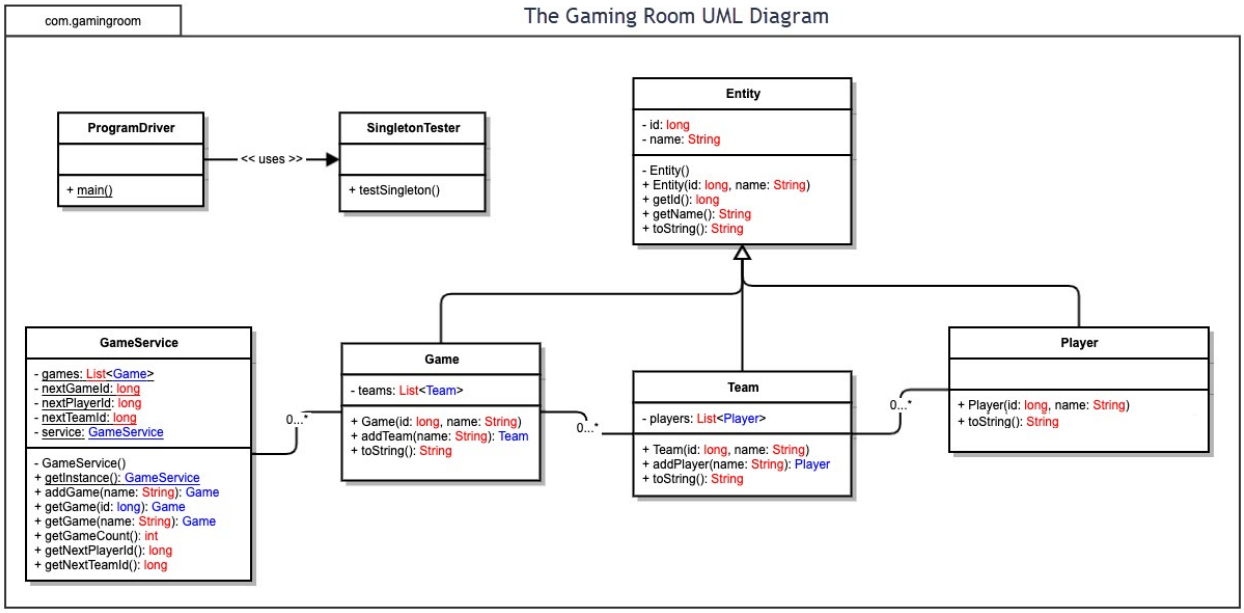
System Architecture View

Please note: There is nothing required here for these projects, but this section serves as a reminder that describing the system and subsystem architecture present in the application, including physical components or tiers, may be required for other projects. A logical topology of the communication and storage aspects is also necessary to understand the overall architecture and should be provided.

Domain Model

The provided Domain Model UML diagram establishes an object-oriented inheritance hierarchy centered around a base class, Entity, from which Game, Team, and Player derive.

- Class Relationships:
  - Entity serves as the abstract superclass holding common attributes: id and name.
  - Game, Team, and Player extend Entity via Inheritance, inheriting the identification attributes and accessor methods.
  - Game maintains a 1 to Many Composition/Aggregation relationship with Team.
  - Team maintains a 1 to Many Composition/Aggregation relationship with Player.
  - GameService acts as a centralized manager holding a collection of Game instances.
- Object Oriented Programming Principles Demonstrated:
  - Inheritance: Extends shared attributes and methods from Entity to Game, Team, and Player eliminating redundant code and standardizing entity behaviors.
  - Encapsulation: Class attributes are marked private or protected and expose strictly through public getter methods and controlled modifier routines, safeguarding internal object state from illegal external mutation.
  - Abstraction and Polymorphism: High level service methods can interact with generic Entity references or collections without needing to know specific subclass execution details enabling clean maintainable code expansion.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS offers high stability and Unix based architecture making it a viable host for smaller development servers. However, its primary weakness for server-side hosting is cost and scalability: Apple hardware is expensive, lacks enterprise rack mount server form factors, and is rarely supported natively by major cloud infrastructure providers for large scale web deployment.	Linux is the industry standard for web application hosting due to its open-source nature, low cost, modularity, and superior performance efficiency. Its primary advantages include enterprise security, minimal overhead, robust command line administration and native support across all major cloud providers. Its main weakness is the requirement for specialized Linux systems administration expertise.	Windows Server provides an enterprise ready hosting environment with strong active directory integration graphical management tools and native support for .NET frameworks. Its advantages include ease of administrative setup for Windows centric IT teams. Its weaknesses include higher licensing costs compared to Linux higher resource overhead and frequent mandatory update cycles requiring server reboots.	Mobile devices are functionally incapable of acting as primary server-side hosts for web applications due to hardware power limits thermal constraints dynamic network IP addressing battery management policies that throttle background processing and limited concurrent connection handling.

Client Side	Supporting macOS web clients requires testing web rendering across Apple's native Safari browser and webkit engine. Software development costs and time are moderate as standard web technologies render consistently. Expertise needed includes cross browser compliance testing and ensuring responsive design for high DPI displays.	Supporting Linux web clients requires minimal extra cost and development time because Linux desktop users rely on standard open-source web browsers. Development expertise requires verifying open web standards and ensuring compatibility with various window managers and font rendering engines without relying on proprietary browser extensions.	Windows represents a massive portion of desktop web clients. Supporting Windows clients involves ensuring web app compatibility across Google Chrome, Microsoft Edge, and Firefox. Development time and cost are low when using standard web frameworks, requiring expertise in responsive design and cross browser testing across varied hardware resolutions.	Supporting mobile device clients requires significant design considerations for responsive user interfaces, touch controls, smaller screen real estate, and varying mobile network speeds. Development costs and time increase due to the need for mobile first web layouts or hybrid wrapper frameworks requiring specialized front end mobile UX expertise.
Development Tools	Relevant languages for building web applications on macOS include Java, JavaScript/TypeScript, Python, and HTML/CSS. Popular IDEs and tools on macOS include Eclipse, IntelliJ IDEA, VS Code, and Xcode. macOS natively includes Unix terminal tools, Git version control, and containerization platforms like Docker Desktop.	Relevant languages for Linux deployment include Java, Python, Node.js, and C++.IDEs and development tools include Eclipse, IntelliJ IDEA, VS Code, Vim, and GNU Compiler Collection. Linux natively excels at automated build scripting, CI/CD pipeline integration, terminal utilities and native Docker containerization.	Relevant languages for building software on Windows include Java, C#, JavaScript, and Python. Primary tools and IDEs include Eclipse, Visual Studio, Visual Studio Code, and IntelliJ IDEA. Windows supports development environments via Git for Windows, PowerShell, and the Windows subsystem for Linux.	Building mobile web client interfaces involves HTML5, CSS3, JavaScript/TypeScript, and frameworks like React, Angular, or Vue. Mobile development tools and IDEs include Android Studio, Xcode, VS Code, and browser developer inspection tools.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**:
   Recommendation: Linux with a Responsive Web Client Architecture

   To expand *Draw It or Lose It* to a multi-platform environment efficiently, The Gaming Room should use Linux based cloud server paired with a web-based client layer.

   Hosting the Java backend on Linux provides maximum cost efficiency, enterprise stability, high concurrent connection throughput, and seamless containerization. Delivering the client via a web application guarantees that users on Windows, macOS, Linux, iOS, and Android can play immediately through any standard web browser without downloading platform specific apps.

2. **Operating Systems Architectures**: Linux uses a monolithic kernel architecture with modular runtime loading capability. It separates system processing cleanly into Kernel Space and User Space. This separation ensures that an application-level error or crash in the web service will not bring down the underlying operating system.

3. **Storage Management**: The recommended operating system uses the ext4 file system for local storage, providing high speed journaling, file system consistency, and efficient block allocation for log files and assets. For application data, the system should pair with a relational database management system or an in-memory cache to store persistent user data, game history, and active session tokens reliably.

4. **Memory Management**: Linux manages physical RAM using Paging and Virtual Memory Management, mapping virtual address spaces to physical memory blocks via page tables. When running the Java based application:

   a. The Java Virtual Machine requests a heap memory pool from the Linux OS.
   b. The JVM manages internal application memory via Garbage Collection automatically allocating memory for active Game, Team, Player objects and reclaiming unreferenced memory.
   c. Linux swap space provides fallback virtual memory on disk to prevent Out of Memory kernel panics during sudden user traffic spikes.

5. **Distributed Systems and Networks**: Communication between distributed web clients and the centralized Linux backend is accomplished using standard web protocols over TCP/IP:
   a. **Transport and Protocol**: Clients communicate with the backend using HTTP/HTTPS RESTful APIs or WebSockets for real time game updates.
   b. **Dependency and Outage Mitigation**:
      i. Network Latency/Disconnects: WebSockets with client-side heartbeats should be used to detect dropped player connections quickly.
      ii. Server Outages: Deploying stateless application nodes behind a Load Balancer ensures that if one server instance fails, traffic is automatically rerouted to a healthy instance without interrupting ongoing games.

6. **Security**:
   To protect user information and secure communications across diverse platforms:

   - Data in Transit: All network communication between web browsers/mobile clients and the server must be encrypted using TLS/SSL, preventing eavesdropping and man in the middle attacks.
   - Authentication and Authorization: Use stateless, encrypted tokens such as JSON Web Tokens or OAuth 2.0 for session verification instead of vulnerable session cookies.
   - Platform Security Features: Leverage Linux kernel security capabilities, including SELinux/AppArmor access controls, configured UFW/iptables firewalls, non-root execution environments for the Java application, and parameterized database queries to prevent SQL injection and cross site scripting vulnerabilities.